Solutions by **Andrew Lys** alys(at)ncsu.edu

Problem 1. Let f take a tensor and return a tensor of a potentially different shape. We can write a residual network for this case as

$$\begin{aligned} L_1 &= A_1(L_0) + f_1(L_0) \\ L_2 &= A_2(L_1) + f_2(L_1) \\ &\vdots \\ L_N &= A_N(L_{N-1}) + f_N(L_{N-1}) \end{aligned}$$

The function A_n adjusts the shape of the previous layer to match the shape of $f_n(L_{n-1})$. Suppose that in a CNN we have that $f_n(L_{n+1})$ has the same spatial dimension but fewer features than L_{n-1}.

Part a. Write a tensor expression for an adjustmetn function A_n in this case where the dimension is reduced by multipying the larger dimension feature vector by a matrix.

Proof. Let A_n be the matrix that reduces the feature dimension. This is a matrix of shape $n_{out} \times n_{in}$ where n_{out} is the number of features in $f_n(L_{n-1})$ and n_{in} is the number of features in L_{n-1}. We may then write the adjustment function as:

$$A_n(L_{n-1})[b, x, y, i] = A[i, J]L_{n-1}[b, x, y, J]$$

□

Part b. Write a tensor expression for reducing the number of features simply by selecting some features and ignoring others.

Proof. Let I be set of indices we want to keep. Then we can write the adjustment function as:

$$A_n(L_{n-1})[b, x, y, I] = L_{n-1}[b, x, y, I]$$

□

Part c. A standard motivation for residual networks is that there is a direct gradient path from the loss to the first layer of the network. Which of the adjustment methods of parts I and II preserve this property?

Proof. The adjustment method of part I preserves this property. This is because the adjustment function is a linear transformation of the input, and thus the gradient can flow through it without any issues. The adjustment method of part II does not preserve this property, because it involves selecting some features and ignoring others, which can lead to a loss of information and thus a loss of the direct gradient path. □

Part d. Does the shape adjustment given on slide 15 of the notes preserve the direct parth from the loss to the first layer?

Proof. No, because only some of the features are selected, and the others are ignored. □

Problem 2. Consider a regression problem where we want to predict a scalar y from a vector x. Consider the L-layer perception for this problem which compute hidden layer vectors $h_1[I], \ldots, h_L[I]$ and predictions $\hat{y}_1, \ldots, \hat{y}_L$ where $\hat{y}_l$ is done by a linear regression on the hidden vector $h_l[I]$

$$\begin{aligned} h_0[i] &= x[i] \\ &\vdots \\ h_{l+1}[i] &= \sigma(W^{h,h}_{l+1}[i, I]h_l[I] - B^{h,h}_{l+1}[i]) \\ \hat{y}_{l+1} &= W^{h,y}_{l+1}[I]h_{l+1}[I] - B^{h,y}_{l+1} \\ &\vdots \\ \text{Loss} &= \sum_{l=1}^{L}(y - \hat{y}_l)^2 \end{aligned}$$

Each term in the loss is called a loss head and defines a loss on each prediction $\hat{y}_l$. Note that there is still only one scalar loss minimized by SGD.

Part a. Explain why these multipe loss heads might improve the ability of SGD to find a useful L-layer MLP regression $\hat{y}_L$ when L is large.

Proof. The multiple loss heads allow gradients to flow directly from the loss to each hidden layer. This allows parameters in earlier layers to change more. □

Part b. As a function of L what is the order of run time for the backpropagation procedure.

Proof. The order of run time is $O(L)$. This is because apart from the gradient of the loss with respect to $\hat{y}_l$, which is $O(L)$, the rest of the backpropagation procedure is $O(1)$ for each assignment. Then the run time of backpropagation is proportional to the number of assignments, which is $O(L)$. □

Part c. Rewrite the above MLP equations to use residual connections rather than multiple heads.

Proof.

$$\begin{aligned}
h_0[i] &= x[i] \\
&\vdots \\
R_{l+1}[i] &= \sigma(W^{h,R}_{l+1}[i, I]h_l[I] - B^{h,R}_{l+1}[i]) \\
h_{l+1}[i] &= W^{h,h}_{l+1}[i, I]h_l[I] - B^{h,h}_{l+1}[i] + R_{l+1}[i] \\
&\vdots \\
\hat{y}_L &= W^{h,y}_L[I]h_L[I] - B^{h,y}_L \\
\text{Loss} &= (y - \hat{y}_L)^2
\end{aligned}$$

□

Problem 3. Consider a single unit defined by

$$u = f(W[I]x[I] - B)$$

Where B is initialized to zero. The vector x is a random variable determined by a random draw of a training example. Suppose that the components of x are independent and $\mathbb{E}\, x[i] = 0$ and $\mathbb{E}\, x[i]^2 = 1$. Suppose we initialize each weight in W independently from a distribution with mean zero, variance σ^2 and is symmetric about zero. Consider $y = \sum_i W[i]x[i]$ as a random variable.

Part a. What value of σ for $W[i]$ gives zero mean and unit variance for y?

Proof.

$$\begin{aligned}
\mathbb{E}\, W[I]x[I] &= \sum_i \mathbb{E}\, W[i]x[i] = \sum_i \mathbb{E}\, W[i]\, \mathbb{E}\, x[i] \\
\mathbb{E}\, W[I]x[I]^2 &= \sum_{i,j} \mathbb{E}\, W[i]x[i]W[j]x[j] = \sum_i \mathbb{E}\, W[i]^2 x[i]^2 \\
&= \sum_i \mathbb{E}\, W[i]^2\, \mathbb{E}\, x[i]^2 = \sum_i \sigma^2 \\
&= d\sigma^2
\end{aligned}$$

Therefore, we choose $\sigma = \frac{1}{\sqrt{d}}$ □

Part b. Suppose that f is sigmoid. What is the mean of u

Proof. Let p be the density of $W[i]x[i]$. We have:

$$\begin{aligned} p(z) &= \int p(z|W[i]=w)p(W[i]=w)dw \\ &= \int p(-z|W[i]=-w)p(W[i]=-w)dw \\ &= p(-z) \end{aligned}$$

Therefore $W[i]x[i]$ is symmetric about zero. Thus, we have:

$$\begin{aligned} \mathbb{E}\, u &= \int f(z)p(z)dz = \int_0^\infty f(z)p(z)dz + \int_{-\infty}^0 f(z)p(z)dz \\ &= \int_0^\infty f(z)p(z)dz + \int_{-\infty}^0 1 - f(-z)p(-z)dz \\ &= \int_0^\infty f(z)p(z)dz + \int_0^\infty 1 - f(z)p(z)dz \\ &= \int_0^\infty p(z)dz = \frac{1}{2} \end{aligned}$$

□

Part c. Compare the variance of u to the variance of y?

Proof. Note the following identity for variance. Let X be some random variable with finite variance, and let X_1, X_2 be i.i.d. copies of X. Then we have:

$$\frac{1}{2}\,\mathbb{E}\,(X_1 - X_2)^2 = \frac{1}{2}\,\mathbb{E}\,X_1^2 - \mathbb{E}[X_1]\,\mathbb{E}[X_2] + \frac{1}{2}\,\mathbb{E}\,X_2^2 = \mathbb{E}\,X^2 - \mathbb{E}[X]^2 = \mathbb{V}\mathrm{ar}(X)$$

Therefore, noting that the sigmoid is a $\frac{1}{4}$-Lipschitz function, we have:

$$\mathbb{V}\mathrm{ar}(u) = \frac{1}{2}\,\mathbb{E}\,(f(y_1) - f(y_2))^2 \le \frac{1}{2}\,\mathbb{E}\,(y_1 - y_2)^2 = \mathbb{V}\mathrm{ar}(y)$$

□

Part d. What is the largest possible variance of the output of a sigmoid?

$$\mathbb{V}\mathrm{ar}(u) = \frac{1}{2}\,\mathbb{E}\,(f(y_1) - f(y_2)^2) = \frac{1}{2}\,\mathbb{E}\,f'(y)^2(y_1 - y_2)^2$$

We want the distribution of y to be symmetric, so that $f'(y) = f'(0) = \frac{1}{4}$ with high probability. Additionally, we want $|y_1 - y_2|$ to be as large as possible, so we choose y to be $\pm\infty$ with probability $\frac{1}{2}$.

With this choice, we have:

$$\begin{aligned} \mathbb{E}\,f(y) &= \frac{1}{2}f(-\infty) + \frac{1}{2}f(\infty) = \frac{1}{2} \\ \mathbb{E}\,(f(y) - \frac{1}{2})^2 &= \frac{1}{2}\left((f(\infty) - \frac{1}{2})^2 + (f(-\infty) - \frac{1}{2})^2\right) = \frac{1}{4} \end{aligned}$$

Problem 4. Counting multiplications in bottle neck layers.

Consider a bottleneck MLP with residual connections defined as follows where N_{bottle} is smaller than $N_{\text{in}} = N_{\text{out}}$.

$$\begin{aligned} \tilde{L}_\ell[n_{\text{bottle}}] &= \mathrm{ReLU}(W_\ell^{b,1}[n_{\text{bottle}}, N_{\text{in}}]L_\ell[N_{\text{in}}] - B_\ell^{b,1}[n_{\text{bottle}}]) \\ \hat{L}_\ell[n_{\text{out}}] &= \mathrm{ReLU}(W_\ell^{b,2}[n_{\text{out}}, N_{\text{bottle}}]\tilde{L}_\ell[N_{\text{bottle}}] - B_\ell^{b,2}[n_{\text{out}}]) \\ L_{\ell+1}[n] &= L_\ell[n] + \hat{L}_\ell[n] \end{aligned}$$

Part a. What is the number of multiplications done by this network as a function of $N_{\text{in}} = N_{\text{out}} = N, N_{\text{bottle}}, L$? Under what condition does this give fewer multiplications than the standard MLP with one matrix between layers?

Proof. To compute $\tilde{L}_\ell[n_{\text{bottle}}]$, we do N_{in} multiplications. Thus, to compute $\tilde{L}_\ell[N_{\text{bottle}}]$ we do $N_{\text{bottle}}N_{\text{in}}$ multiplications. For each $\hat{L}_\ell[n_{\text{out}}]$, we do N_{bottle} multiplications, so to compute $\hat{L}_\ell[N_{\text{out}}]$ we do $N_{\text{bottle}}N_{\text{out}}$ multiplications. Thus, to compute $L_{\ell+1}[N]$ we do $N_{\text{bottle}}N_{\text{in}} + N_{\text{bottle}}N_{\text{out}} = 2N_{\text{bottle}}N$ multiplications. Therefore, to compute $L_L[N]$ we do $2N_{\text{bottle}}NL$ multiplications.

A standard MLP with one matrix is given by:

$$L_{\ell+1}[n] = \text{ReLU}(W_\ell[n, N]L_\ell[N] - B_\ell[n])$$

So to compute $L_{\ell+1}[n]$ we do N^2 multiplications, and to compute $L_L[n]$ we do N^2L multiplications. Thus, the bottleneck MLP gives fewer multiplications when

$$2N_{\text{bottle}}NL < N^2L \implies N_{\text{bottle}} < \frac{N}{2}$$

□

Part b. Consider γ into the residual connection

$$L_{\ell+1}[n] = \gamma(L_\ell[n] + \hat{L}_\ell[n])$$

If the network is initialized such that each response of $L_\ell[n]$ and $\hat{L}_\ell[n]$ as mean zero and unit variance, and are independent, what value of γ gives that $h[\ell + 1, j]$ has zero mean and unit variance.

Proof.

$$\mathbb{V}\text{ar}(L_\ell[n] + \hat{L}_\ell[n]) = \mathbb{V}\text{ar}(L_\ell[n]) + \mathbb{V}\text{ar}(\hat{L}_\ell[n]) = 2$$

Therefore $\gamma = \frac{1}{\sqrt{2}}$ suffices. □

Part c. Why might $\gamma < 1$ be damaging to the optimization of the lower layers of the residual stack?

Proof. If $\gamma < 1$, then we have vanishing gradients. □

Problem 5. RNN run time. Consider an autoregressive RNN neural language model with

$$\Pr_\Phi(w_t|w_1, \ldots, w_{t-1}) = \underset{w'}{\text{softmax}} e[w', I]h[t-1, I]$$

$e[w, I]$ is the word vector for word w and $h[t, I]$ is the hidden state vector at time t of a left-to-right RNN. For the first word w_1, we have an externally provided initial hidden state $h[0, I]$ and $w_1, \ldots, w_0$ denotes the empty string. We train the model on the full loss:

$$\begin{aligned}\Phi^* &= \arg\min_\Phi \mathop{\mathbb{E}}_{w_1,\ldots,w_T\sim\text{Train}} -\log \Pr_\Phi(w_1, \ldots, w_T) \\ &= \arg\min_\Phi \mathop{\mathbb{E}}_{w_1,\ldots,w_T\sim\text{Train}} \sum_{t=1}^{T} -\log \Pr_\Phi(w_t|w_1, \ldots, w_{t-1})\end{aligned}$$

What is the order of run time as a function of sentence length T for the backpropagation for this model run on a sentence $w_1, \ldots, w_T$?

Proof. It is $O(T)$, since the length of the assignment program is of the order $O(T)$. □